

Namaste dosto! Main aapka Senior AI Engineer aur LangChain Instructor. Welcome to **Phase 3: Advanced RAG**.

Aapne basic RAG master kar liya hai (Loaders, Splitters, Embeddings, Vector DBs, aur LLM chains). Lekin production mein, real-world applications fail ho jaati hain agar hum basic RAG use karein. Kyun? Kyunki "garbage in, garbage out." Agar context sahi nahi aayega, toh LLM hallucinate karega.

Aaj se hum industry-level Advanced RAG techniques seekhenge, shuru karte hain **Module 1** se.

Module 1: Parent Document Retriever

1. Why this concept exists

Jab hum RAG banate hain, humein accuracy (exact match) aur context (broad understanding) dono chahiye hote hain. Parent Document Retriever isiliye exist karta hai taaki hum search chhote (small) chunks par karein for high accuracy, but LLM ko bada (large) chunk bhejin for complete context.

2. Problem with traditional RAG

Basic RAG mein ek sabse badi problem hoti hai **Chunk Size Dilemma**:

- **Small Chunks (e.g., 200 tokens):** Similarity search bahut accurate hoti hai, exact answer mil jata hai. *Lekin*, aas-paas ka context missing hota hai, jisse LLM confuse ho jata hai.
- **Large Chunks (e.g., 2000 tokens):** Context toh poora mil jata hai, *lekin* vector search ki accuracy gir jaati hai kyunki chunk ke andar bahut saara irrelevant noise hota hai (jisko "Lost in the middle" problem bhi kehte hain).

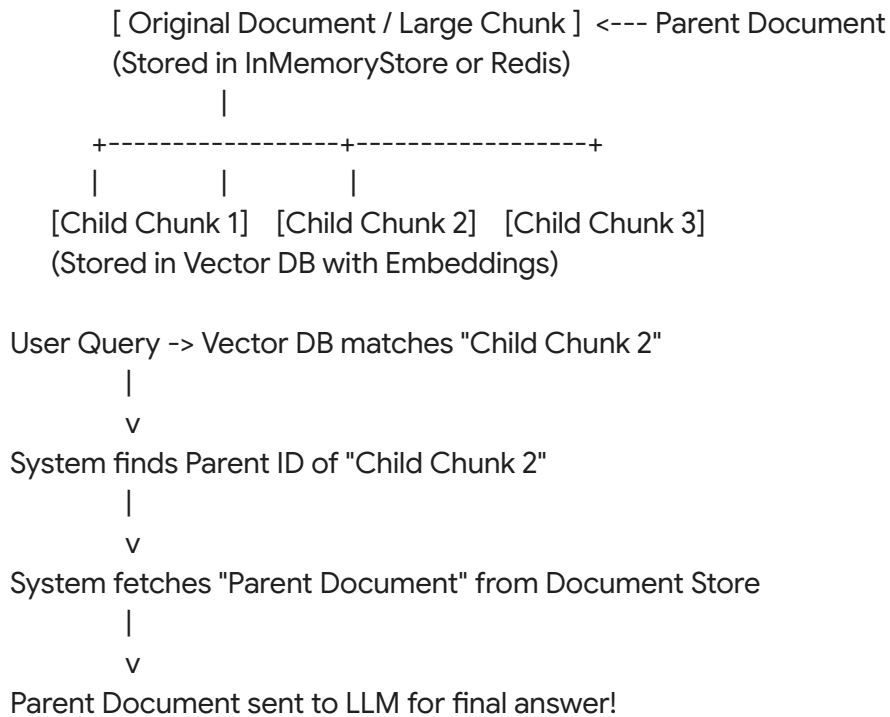
3. Real-world example

Maan lo aap ek Legal Contract padh rahe ho.

Aapne search kiya: "*Termination penalty kitni hai?*"

- **Small chunk match:** "The penalty will be \$10,000." (Aapko amount mil gaya, par terms & conditions nahi mili).
- **Large chunk match:** Pura 5-page ka document aagaya, jisme termination penalty ka zikr hi dab gaya.
- **Parent-Child Solution:** Hum search us choti line (Child) par karenge "The penalty will be \$10,000", lekin jab match ho jayega, toh hum LLM ko poora "Termination Clause" (Parent paragraph) utha kar de denge!

4. Visual explanation using diagrams made with text



5. Architecture flow

1. Hum apne documents ko **Large Chunks** (Parent Documents) mein split karte hain.
2. Un Parent Documents ko hum ek **Document Store** (jaise InMemoryStore ya MongoDB) mein save karte hain aur unko ek unique ID assign karte hain.
3. Fir hum un Parent Documents ko aur chhote **Small Chunks** (Child Documents) mein split karte hain.
4. Har Child Document ke metadata mein uske Parent ka ID daal kar, usko embed karke **Vector Database** (Chroma/FAISS) mein save karte hain.
5. Jab user question poochta hai, similarity search **Child Documents** par hoti hai.
6. Jo top child documents match hote hain, unke IDs se **Parent Documents** fetch kiye jaate hain.
7. Parent Docs LLM ko as context pass hote hain.

6. Step-by-step implementation

1. Vector DB (Chroma) initialize karo.
2. Parent Text Splitter (bade chunks) aur Child Text Splitter (chhote chunks) banao.
3. Ek InMemoryStore initialize karo parent docs rakhne ke liye.
4. ParentDocumentRetriever initialize karo.
5. Documents add karo (Retriever automatically unko parent/child mein split karke vector DB aur store mein save kar dega).
6. Chain banakar query karo.

7. Complete Python code & LangChain implementation

Chaliye isko code karte hain. (Make sure you have langchain, langchain-chroma, langchain-openai installed).

```
import os
from langchain_community.document_loaders import PyPDFLoader
from langchain_text_splitters import RecursiveCharacterTextSplitter
from langchain_openai import OpenAIEmbeddings, ChatOpenAI
from langchain_chroma import Chroma
from langchain.storage import InMemoryStore
from langchain.retrievers import ParentDocumentRetriever
from langchain.chains import create_retrieval_chain
from langchain.chains.combine_documents import create_stuff_documents_chain
from langchain_core.prompts import ChatPromptTemplate

# 1. Setup API Keys
os.environ["OPENAI_API_KEY"] = "sk-your-openai-api-key"

# 2. Load PDF Document
loader = PyPDFLoader("company_policy.pdf")
docs = loader.load()

# 3. Setup Text Splitters
# Parent Splitter: Bade chunks (Context ke liye)
parent_splitter = RecursiveCharacterTextSplitter(
    chunk_size=2000,
    chunk_overlap=200
)

# Child Splitter: Chhote chunks (Search accuracy ke liye)
child_splitter = RecursiveCharacterTextSplitter(
    chunk_size=400,
    chunk_overlap=50
)

# 4. Setup Vector DB & Document Store
```

```

embeddings = OpenAIEmbeddings()
vectorstore = Chroma(
    collection_name="split_parents",
    embedding_function=embeddings
)
# Ye InMemoryStore hamare bade Parent Chunks ko hold karega
store = InMemoryStore()

# 5. Initialize ParentDocumentRetriever
retriever = ParentDocumentRetriever(
    vectorstore=vectorstore,
    docstore=store,
    child_splitter=child_splitter,
    parent_splitter=parent_splitter,
)

# 6. Add documents to the retriever
# Yeh automatically dono splits banayega aur vector/doc store mein daal dega
retriever.add_documents(docs)

# Aap check kar sakte hain ki vector DB mein chhote chunks hain aur store mein bade!
print(f"Number of parent docs in store: {len(list(store.yield_keys()))}")

# 7. Create RAG Chain
llm = ChatOpenAI(model="gpt-3.5-turbo", temperature=0)

prompt = ChatPromptTemplate.from_template("""
Answer the question based only on the following context:
{context}

Question: {input}
""")

document_chain = create_stuff_documents_chain(llm, prompt)
retrieval_chain = create_retrieval_chain(retriever, document_chain)

# 8. Query the System
user_query = "What is the policy on remote work setup allowance?"
response = retrieval_chain.invoke({"input": user_query})

print("\n--- Answer ---")
print(response["answer"])

# Agar aap response["context"] print karenge, toh aap dekhenge

```

ki pura parent chunk aaya hai, na ki sirf 400 characters ka child chunk.

8. Production use case

Enterprise HR Chatbot: HR policies bahut detailed hoti hain. Agar koi employee "maternity leave days" search kare, toh vector search sirf "90 days" wala sentence match karegi. Lekin agar hum wo sentence LLM ko bhejenge, toh "conditions and eligibility criteria" miss ho jayenge. Parent Document Retriever ensure karta hai ki "90 days" (child) match hone par poora "Maternity Leave Section" (parent) LLM ke paas jaye, taaki employee ko 100% correct aur contextual policy batai ja sake.

9. Interview questions

1. **Q: How does ParentDocumentRetriever solve the context vs accuracy trade-off?**
A: By decoupling retrieval from generation. It uses small chunks for high-accuracy semantic search (retrieval) but passes the larger parent chunks to the LLM for generation (context).
2. **Q: Can the "Parent" just be the entire original document?**
A: Yes! Agar aap parent_splitter define nahi karte, toh wo default original un-split document ko hi parent maan lega. Lekin badi PDFs mein this can exceed the LLM context window.
3. **Q: Where are parent documents stored? In the vector database?**
A: No. Vector database sirf child chunks aur embeddings store karta hai. Parent documents key-value format (Document Store) mein store hote hain jaise Redis ya InMemoryStore.

10. Advantages

- **High Precision:** Search bahut exact hoti hai kyunki child chunks chhote hain.
- **Rich Context:** LLM hallucinate nahi karta kyunki usko poora surrounding context (parent) milta hai.
- **Better QA:** Complex multi-hop questions jisme aas-paas ki info chahiye, wo asani se solve ho jate hain.

11. Limitations

- **Increased Token Usage:** Har baar bada document LLM ko jayega, jisse API cost thodi badh sakti hai.
- **Storage Overhead:** Data do jagah store ho raha hai (Vector DB mein bachhe, Doc Store mein parents).
- **Retrieval Latency:** Ek extra step add ho gaya (fetching from doc store via ID).

12. Best practices

- **Chunk Sizes:** Child chunks ko 200-400 characters ke aas-paas rakhein. Parent chunks ko 1000-2000 rakhein.

- **Docstore Choice:** Production ke liye InMemoryStore use mat karna (server restart hone par data chala jayega). RedisStore ya MongoDB ka use karna.
- **Overlap:** Child chunks mein thoda overlap zaroor dena taaki context edge par break na ho.

Assignments & Practical Exercises

- **Exercise 1:** Ek choti 10-page ki story PDF download karo. Uspar normal VectorStoreRetriever aur ParentDocumentRetriever dono run karo aur output compare karo. Notice karo ki Parent wala kitne ache se story ka pichla background include karta hai!
- **Exercise 2:** Code mein jahan InMemoryStore() use hua hai, LangChain docs padh kar usko LocalFileStore ya kisi persistent store mein convert karne ka try karo.

Common Mistakes Beginners Make

- *Parent ko LLM ki context limit se bada rakh dena:* Agar parent chunk size 10,000 rakh diya aur 4 chunk retrieve kar liye, toh 40k tokens ho jayenge aur API rate limit / context limit exceed error de degi.
- *Forgetting to add metadata:* Custom implementations mein bache hue chunks mein parent ki ID metadata mein attach karna bhool jaana. (LangChain ki class ye khud handle kar leti hai).

Toh yeh tha hamara Module 1: Parent Document Retriever. Samajh aaya aapko ki context kitna zaroori hai?